



< Previous



Next >

## Complexity and Structure

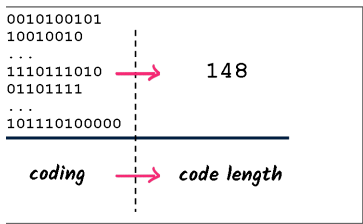
🔖 Bookmark this page

Problem: Can we define the complexity of new objects, based on their structure?

as.

```
, size(4),  
) , size(4),
```

an then be converted  
eans) into binary  
is binary form gives  
plexity of the object.



f estimating  
on consists of  
. . . .

For now, we are able to estimate the complexity of integers and of objects that are stored in memory (through the complexity of their address). What about new objects?

Consider, for instance, the case of passwords. When looking for a good password, we try to outsmart machines. This situation offers a nice illustration of the importance of complexity.

**Question:** Complex passwords are impossible to remember. Can we define what a good password should be?

**Answer:** A good password should be easy to remember, but hard to guess. In AIT’s terms, it should be *complex* to anyone but you.

Password complexity

6/6 points (ungraded)

How would you *rank* the complexity of the following passwords (from 1=simplest to 6=most complex)?

efghijkl

3

✓ Answer: [435]

aabbccdd

5

✓ Answer: [534]

abcdefgh

2

✓ Answer: [532]

amoijsdf

6

✓ Answer: 6

aaaaaaaa

1

✓ Answer: 1

abcdabcd

4

✓ Answer: [^16]

The ranking that is most often proposed is (from simplest to most complex):

1. aaaaaaaaa

2. abcdefgh

3. abcdabcd

4. efghijkl

5. aabbccdd

6. amoijsdfs

Note that though there is no "objective" good ranking, we expect aaaaaaaaa to come first and amoijsdfs to be last.

Submit

You have used 1 of 1 attempt

**i** Answers are displayed within the problem

Passwords are character strings. Some strings are obviously simpler than others. How can we quantify this? A first step consists of finding a compact formal representation of the string, such as `repetition(a, 8)`, and then translate this formal representation into bits using some appropriate code, and finally counts the bits.

String Structures

Representing String Structures

kkkkkkkk → 1 1 11 1011 1000 C = 12

mnoqrst → 1 10 1 11 1101 1000 C = 14

abcdabcd → 1 1 1 10 1 11 1 100 10 C = 14

aabbccdd → 1 11 1 10 1 11 1 100 1 10 C = 16

Note: here, codes are supposed to be separated by blank characters (as in the "Morse" code)

These codes provide estimates of their complexity.

To get the complexity of the overall tree, we add up the complexity of its elements.

The numbers we get can be used to decide whether some strings are simpler than others.

Conclusion:

The complexity of objects can be assessed by representing them as structures, and then by imagining compact codes to represent

► String Structures in a nutshell

Simple Structure 1

1 point possible (ungraded)

Using the coding scheme presented in the video, determine which string is represented by: `mapping(increment(1,a,4), increment(1,X,2))`.

☐ cdef

☐ aabbccdd

☐ aaaaaaaa

☐ abbccdde

☐ abcdabcd

☐ ee

Submit

You have used 0 of 1 attempt

Simple Structure 2

1 point possible (ungraded)

Provide an expression that represents `abcdefgh`.

Submit

You have used 0 of 1 attempt

| <i>letter</i> | <i>position</i> | <i>code</i> |
|---------------|-----------------|-------------|
| a             | 1               | 0           |
| b             | 2               | 1           |
| c             | 3               | 00          |
| d             | 4               | 01          |
| e             | 5               | 10          |
| f             | 6               | 11          |
| g             | 7               | 000         |
| h             | 8               | 001         |

Suppose we want to measure the complexity of a password such as `abcdefgh`. We can code for the letters one by one. The codes that appear in the video assume standard binary code. We can get a slightly more compact code using our compact integer coding scheme (see table).

If we add up all the bits used in codes, we get a total of 16 bits for `abcdefgh` (not counting space delimiters). But we can do better than that, if we consider that `abcdefgh` can be represented as `increment(1, a, 8)`. To do so, suppose that we limit ourselves to the following list of available operators, with the corresponding codes.



|           |            |           |         |
|-----------|------------|-----------|---------|
| operator: | repetition | increment | mapping |
| code:     | 0          | 1         | 00      |

Suppose now that we consider three kinds of entities, coded accordingly:

|         |          |         |        |
|---------|----------|---------|--------|
| entity: | operator | integer | letter |
| code:   | 0        | 1       | 00     |

Using this coding scheme, an object like letter  $e$  will be coded as 00 10, where the first two bits designate `letters` and the last two bits designate the rank of  $e$  among letters.

Using the code 1

1 point possible (ungraded)

What would be the correct code to designate number 6 in our small language?

☐ 000

☐ 110

☐ 1 000

☐ 1 110

☐ 00 000

☐ 11 110

Submit

You have used 0 of 1 attempt

Using the code 2

1 point possible (ungraded)

What would be the correct code to designate operator `mapping` in our small language?

☐ 0 00

☐ 0 0 00

☐ 000

☐ 00

Submit

You have used 0 of 1 attempt

Using the code 3

1 point possible (ungraded)

What would be the correct code to designate letter `b` in our small language?

☐ 10

☐ 00 11

☐ 00 1

☐ 00

Submit

You have used 0 of 1 attempt

Since we are able to represent `abcdefgh` as `increment (1, a, 8)`, we can now translate it into a binary representation. We still use our compact integer coding system. Note that when the type of the object is known, we can avoid using a complete code and thus get an overall shorter representation.

| <i>element</i> | <i>code</i> | <i>comment</i>                                        |
|----------------|-------------|-------------------------------------------------------|
| _____          | _____       | _____                                                 |
| increment      | 0 1         | 1 = designation of <code>increment</code> as operator |
| 1              | 1           | an integer is expected                                |
| a              | 00 0        | complete code is expected (here: letter + a)          |
| 8              | 010         | an integer is expected                                |
| _____          | _____       | _____                                                 |

We end up with the following code: `0 1 1 00 0 010` to represent `abcdefgh`. It is 9 bits long (ignoring space delimiters), which is more compact than the 16 bits of the letter-by-letter code (note: the code is decodable only if we keep the spaces between code words). Our coding scheme is sufficient to provide an estimate of the description complexity of alphabetic strings.

Description length of `abcdabcd`

1 point possible (ungraded)

Using the same coding method, provide a binary representation of `abcdabcd` ?

Submit

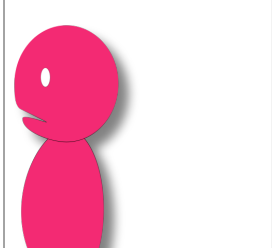
You have used 0 of 1 attempt

Note that the resulting description length of `abcdabcd` is larger than the description length of `abcdefgh`. This might explain why `abcdabcd` is perceived as more complex than `abcdefgh`.





So complex  
integers must be  
far away from  
simple ones?



< Previous

Next >

 Some Rights Reserved



## edX

[About](#)  
[Affiliates](#)  
[edX for Business](#)  
[Open edX](#)  
[Careers](#)  
[News](#)

## Legal

[Terms of Service & Honor Code](#)  
[Privacy Policy](#)  
[Accessibility Policy](#)  
[Trademark Policy](#)  
[Sitemap](#)  
[Cookie Policy](#)  
[Your Privacy Choices](#)

## Connect

[Idea Hub](#)  
[Contact Us](#)  
[Help Center](#)  
[Security](#)

